

Species Identification Machine Learning Working Manual

by

Taposh Mollick



1. Introduction

This working manual has been developed to support data administrators, analysts, and users of the New York Natural Heritage Program (NYNHP) in effectively utilizing the newly developed invasive species identification machine learning (ML) tool. The tool is designed to streamline the process of confirming unconfirmed species observations submitted to the iMapInvasives platform by using ML with geospatial technologies. The primary purpose of this tool is to assist invasive species data managers in accurately identifying species from submitted images, enhancing the reliability and efficiency of data curation across New York State. This manual provides clear guidance on installing, operating, and maintaining the tool, ensuring users can navigate each stage of the workflow from image preprocessing to species confirmation with confidence. This manual is designed to be both comprehensive and user-friendly, ensuring that all stakeholders, from field data managers to GIS specialists, can maximize the tool's potential in monitoring and managing invasive species data in New York.

2. Installation

2.1. Anaconda distribution

All development and execution of python code for this project were conducted within Jupyter Notebook. Users can access Jupyter Notebook for Python coding and utilize the Anaconda PowerShell Prompt for command-line operations directly through the Anaconda Navigator interface. To install Anaconda, visit <https://www.anaconda.com/download> and select the appropriate version for Windows, macOS, or Linux. Follow the installation instructions provided on the site to complete the setup.

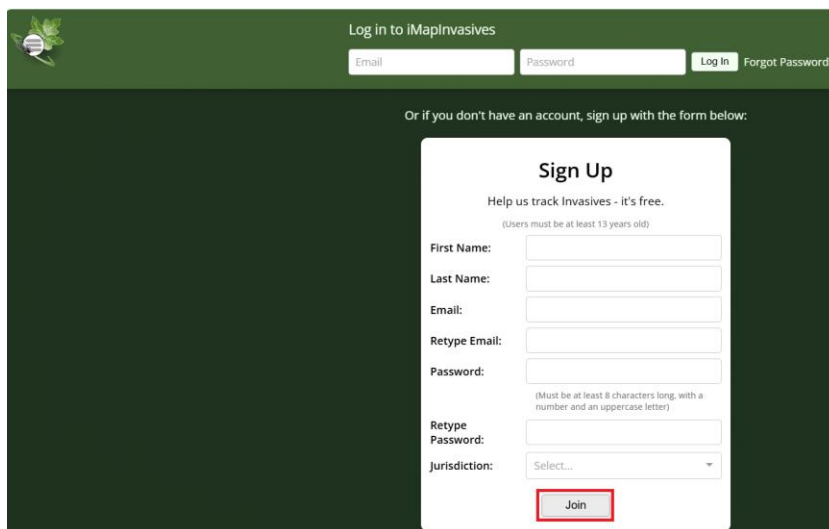
2.2. git

It is recommended to install the Git application to clone the complete codes and datasets from the GitHub repository efficiently. Alternatively, users may choose to download individual files directly from the repository at: <https://github.com/tmollick95/iMapML>. Git can be downloaded and installed for Windows, macOS, or Linux from the official website: <https://git-scm.com/downloads>.

3. Account Creation

3.1. iMapInvasives account

An iMapInvasives account is required to access the iMap API for retrieving both confirmed and unconfirmed species observation records. To create an account, visit <https://imapinvasives.natureserve.org/imap/login.jsp> and complete the sign-up process by selecting the "Join" option and providing the necessary information. Once registered, use your credentials to log in to the iMap web application or authenticate iMap API requests.

The image shows a web page for iMapInvasives. At the top, there is a green header with a logo on the left and a login section on the right. The login section has a title "Log in to iMapInvasives", an email input field, a password input field, a "Log in" button, and a "Forgot Password?" link. Below the header, there is a dark green background with a white sign-up form in the center. The form is titled "Sign Up" and has a subtitle "Help us track Invasives - it's free." followed by a note "(Users must be at least 13 years old)". The form contains several input fields: "First Name:", "Last Name:", "Email:", "Retype Email:", "Password:", "Retype Password:", and a "Jurisdiction:" dropdown menu. A "Join" button is at the bottom of the form, highlighted with a red rectangle. A note next to the password fields states "(Must be at least 8 characters long, with a number and an uppercase letter)".

3.2. RapidAPI account

RapidAPI is a developer platform that provides access to a wide range of APIs, including the iNaturalist VisionAPI used in this project. To create an account, visit <https://rapidapi.com/auth/signup> and register using your email address and create a password. Once your account is active, log in and search for "VisionAPI." If the iNaturalist VisionAPI is not visible, contact the iNaturalist team to request access. The Basic plan allows up to 500 API calls per month at no additional charge; any

further usage requires a paid plan subscription.

< VisionAPI

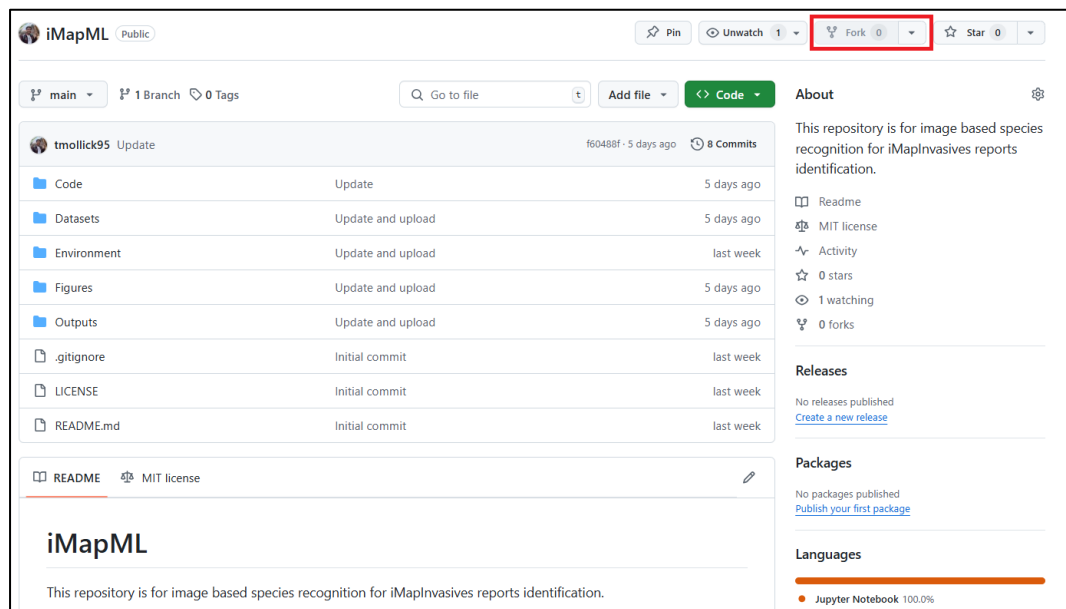
Choose the plan that's right for nyimapiinvasives

RapidAPI partners directly with API providers to give you no-fuss, transparent pricing.

Basic	Pro	Pro (Deprecated)	Ultra	Mega
\$0.00 /mo	\$20.00 /mo	\$20.00 /mo	\$50.00 /mo	\$100.00 /mo
Requests 500 / Month Hard Limit	Requests 3,000 / Month Hard Limit	Requests 2,000 / Month Hard Limit	Requests 10,000 / Month Hard Limit	Requests 20,000 / Month Hard Limit
Rate Limit 1000 requests per hour	Rate Limit 60 requests per minute	Rate Limit 60 requests per minute	Rate Limit 60 requests per minute	Rate Limit 60 requests per minute
Downgrade Plan	Choose This Plan	Cancel Plan	Upgrade Plan	Upgrade Plan

4. Download Codes and Datasets

All code files and datasets can be downloaded collectively or individually from the project's GitHub repository: <https://github.com/tmollick95/iMapML>. To save a copy of the repository to your own GitHub account, simply click the "Fork" button on the repository page. This allows you to maintain your own version for further development or customization.



To begin, open the Anaconda PowerShell Prompt by searching for it in your computer's search bar and clicking to launch the terminal. Navigate to the directory where you want to store the code and datasets by using the following command:

```
cd .\location\
```

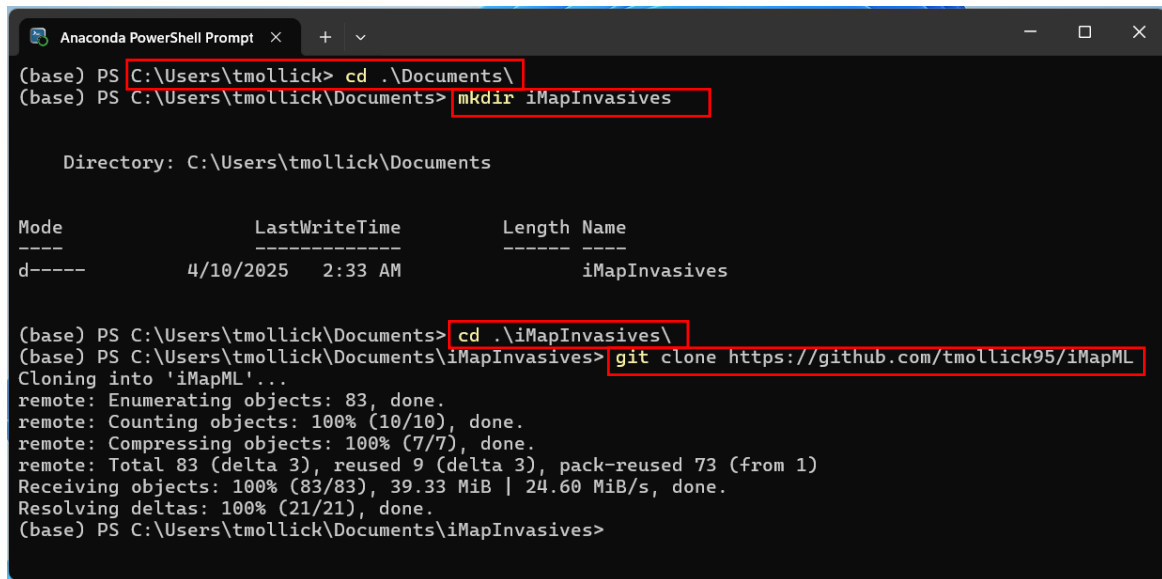
Next, create a new folder to organize the downloaded files:

```
mkdir folder_name  
cd .\folder_name\
```

To download the complete repository to your local machine, run the following command, replacing the URL with the actual GitHub repository link:

```
git clone "GitHub repository link"
```

The actual code is shown in the screenshot below of bash code:



```
Anaconda PowerShell Prompt x + v
(base) PS C:\Users\tmlollick> cd .\Documents\
(base) PS C:\Users\tmlollick\Documents> mkdir iMapInvasives

Directory: C:\Users\tmlollick\Documents

Mode                LastWriteTime         Length Name
----                -
d-----          4/10/2025   2:33 AM                iMapInvasives

(base) PS C:\Users\tmlollick\Documents> cd .\iMapInvasives\
(base) PS C:\Users\tmlollick\Documents\iMapInvasives> git clone https://github.com/tmollick95/iMapML
Cloning into 'iMapML'...
remote: Enumerating objects: 83, done.
remote: Counting objects: 100% (10/10), done.
remote: Compressing objects: 100% (7/7), done.
remote: Total 83 (delta 3), reused 9 (delta 3), pack-reused 73 (from 1)
Receiving objects: 100% (83/83), 39.33 MiB | 24.60 MiB/s, done.
Resolving deltas: 100% (21/21), done.
(base) PS C:\Users\tmlollick\Documents\iMapInvasives>
```

5. Creating a Python Virtual Environment

A Python virtual environment is essential for isolating project-specific dependencies, ensuring that packages required for one project don't interfere with others. It enables reproducibility by allowing users to recreate the exact environment using files like `environment.yml`. This is especially important for collaborative projects or deploying tools like the iMapInvasives species identification system, where consistency across systems is critical. To create a virtual environment using Anaconda, follow the official documentation: <https://docs.conda.io/projects/conda/en/latest/user-guide/tasks/manage-environments.html> to create a new virtual environment using the `environment.yml` file.

Follow these steps in the Anaconda PowerShell Prompt:

- a) Navigate to the directory containing your `environment.yml` file:

```
cd path/to/your/environment.yml
```

b) Create the environment using the environment.yml file:

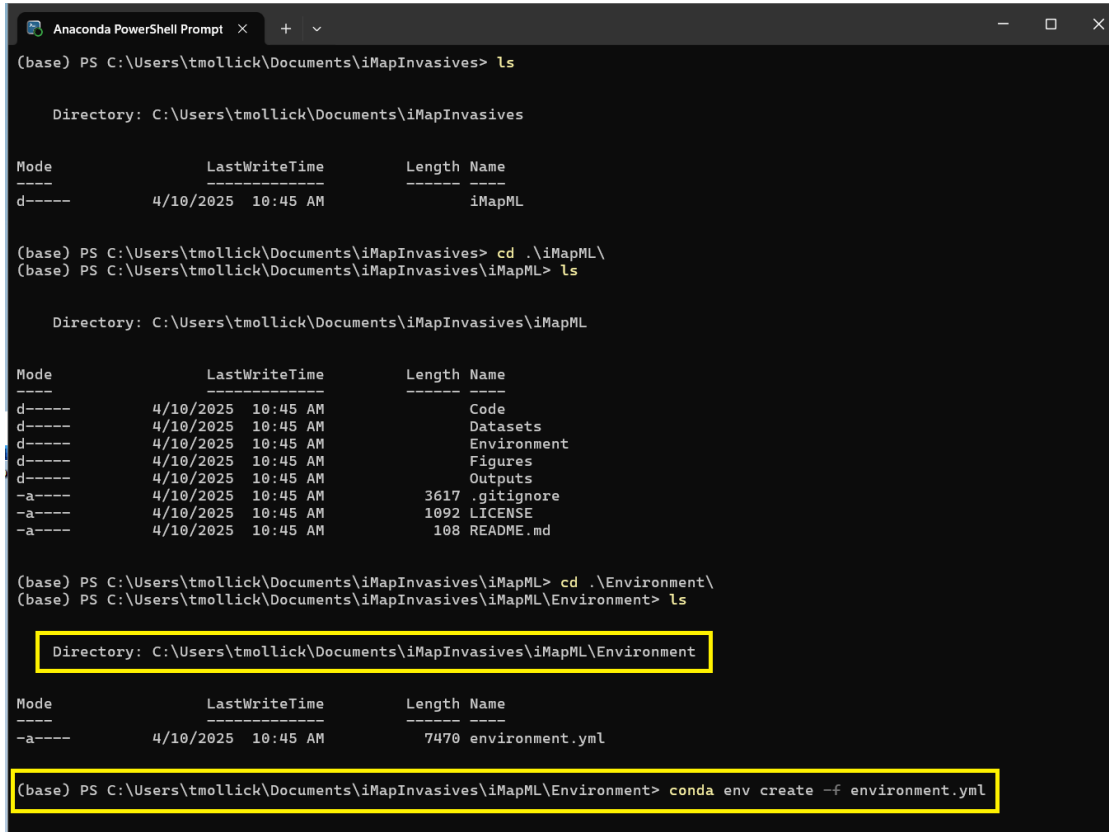
```
conda env create -f environment.yml
```

c) Activate the new environment:

```
conda activate iMapInvasives
```

d) Verify that the environment was successfully created:

```
conda env list
```

A screenshot of the Anaconda PowerShell Prompt window. The window title is "Anaconda PowerShell Prompt". The terminal shows the following commands and output:
(base) PS C:\Users\tmollick\Documents\iMapInvasives> ls

Directory: C:\Users\tmollick\Documents\iMapInvasives

Mode LastWriteTime Length Name
---- -
d----- 4/10/2025 10:45 AM iMapML

(base) PS C:\Users\tmollick\Documents\iMapInvasives> cd .\iMapML\
(base) PS C:\Users\tmollick\Documents\iMapInvasives\iMapML> ls

Directory: C:\Users\tmollick\Documents\iMapInvasives\iMapML

Mode LastWriteTime Length Name
---- -
d----- 4/10/2025 10:45 AM Code
d----- 4/10/2025 10:45 AM Datasets
d----- 4/10/2025 10:45 AM Environment
d----- 4/10/2025 10:45 AM Figures
d----- 4/10/2025 10:45 AM Outputs
-a----- 4/10/2025 10:45 AM 3617 .gitignore
-a----- 4/10/2025 10:45 AM 1092 LICENSE
-a----- 4/10/2025 10:45 AM 108 README.md

(base) PS C:\Users\tmollick\Documents\iMapInvasives\iMapML> cd .\Environment\
(base) PS C:\Users\tmollick\Documents\iMapInvasives\iMapML\Environment> ls

Directory: C:\Users\tmollick\Documents\iMapInvasives\iMapML\Environment

Mode LastWriteTime Length Name
---- -
-a----- 4/10/2025 10:45 AM 7470 environment.yml

(base) PS C:\Users\tmollick\Documents\iMapInvasives\iMapML\Environment> conda env create -f environment.yml

Add the Conda environment iMapInvasives kernel to Jupyter with the following bash command in

Anaconda PowerShell Prompt:

```
python -m ipykernel install --user --name=iMapInvasives --  
display-name="Python (iMapInvasives) "
```

Now Open Jupyter Notebook to check the kernel:

Go to **Kernel** and select **iMapInvasives** kernel from the drop-down menu.

6. Required python libraries

You need to import the Python libraries into the table below for this project:

Imported Python Libraries	Description
import os	Interact with the operating system (e.g., file paths)
import pandas as pd	Data manipulation and analysis
import numpy as np	Numerical operations and arrays
import matplotlib.pyplot as plt	Plotting and visualizations
%matplotlib inline	Display plots inline in Jupyter Notebook

<code>import seaborn as sns</code>	Advanced statistical data visualization
<code>import requests</code>	Send HTTP requests to access online resources
<code>from PIL import Image</code>	Open and process image files
<code>from io import BytesIO</code>	Handle binary data from streams
<code>import math</code>	Perform mathematical operations
<code>import json</code>	Read and write JSON data
<code>from IPython.display import display, HTML</code>	Display rich content (HTML, tables) in Jupyter
<code>import time</code>	Time-related functions (e.g., delays)
<code>import pickle</code>	Save and load Python objects
<code>import urllib</code>	Open and read URLs
<code>from time import sleep</code>	Pause code execution for a given time
<code>import scipy.stats as stats</code>	Statistical functions and tests
<code>from sklearn.metrics import roc_curve, auc, confusion_matrix, classification_report, precision_recall_curve</code>	Evaluation metrics
<code>from sklearn.linear_model import LogisticRegression</code>	Logistic regression model
<code>from sklearn.model_selection import train_test_split</code>	Split data into training and test sets
<code>from sklearn.model_selection import GridSearchCV</code>	Hyperparameter tuning using grid search
<code>From openpyxl import load_workbook, Workbook</code>	Open the Excel file
<code>Import ipywidgets as widgets</code>	To select the records interactively

7. Jupyter Notebook Python files

7.1. Data Preprocessing and Machine Learning for image recognition for iMapInvasives

Download “Machine learning image recognition for iMapInvasives.ipynb” notebook file for data preprocessing. The notebook is part of the iMapML workflow and is used to filter, clean, and prepare invasive species observation data for further machine learning-based analysis. The focus is on selecting records of the top invasive species from iMap unconfirmed species. Run the notebook cell-by-cell by pressing “Shift + Enter” and monitor the progress.

General Workflow and Instructions for Use:

- Import Required Libraries:** Loads of essential Python packages such as pandas, numpy, matplotlib, seaborn, and requests for data manipulation and visualization.
- Load “**14 Species for Taxon ID and jurisdiction species ID.xlsx**” dataset to get the selected invasive species of interest, their common name, scientific name, jurisdiction species ID, Taxon ID, combined score thresholds, and geo score thresholds.

- c) **Create an interactive selection of widgets:** These widgets enable users to select a common name for the invasive species of interest, confirm or unconfirm record types, specify the number of records, and set an offset to skip those records. Widgets make this notebook user-friendly for non-coders.
- d) **ArcGIS RESTful API service for iMapInvasives:** This API enables the direct selection of iMap records from the database. Generate a token first to access the API. Use your iMap username and password to authenticate the API.
- e) **Create a dataframe of the selected iMap records:** Create a dataframe using the accessed iMap records by the ArcGIS RESTful API and filter the records that fall in the New York jurisdiction, have photos uploaded by the users. It also excluded the records that are Google Street View photos from Daina Krummins.
- f) **Access the iMap API:** Use your iMapInvasives username and password to authenticate and retrieve image URLs linked to each presence ID.
- g) Check the first 10 selected presence ID's photos uploaded by the users of iMapInvasives by plotting.
- h) **Access the VisionAPI Using RapidAPI Key:** Subscribe to iNaturalist VisionAPI on <https://rapidapi.com>. Get your X-RapidAPI-Key and include it in your request headers:

```
headers = {  
    "X-RapidAPI-Key": "your_api_key",  
    "X-RapidAPI-Host": "inaturalist-open-api.p.rapidapi.com"  
}
```

The image is posted to the API, and the top predicted species and confidence scores are returned.

- i) **Apply NYS Invasive Species Tier System and Thresholds:** The predicted species are checked against New York State's invasive species Tier list to assess priority. A combined score is calculated using VisionAPI confidence, geo score (location-based likelihood), and iMap observation metadata. Threshold filters are applied to classify observations.
- j) **Save Output in Excel with HTML View Link:**

The final predictions are saved in an Excel file.

This file includes columns: presence_id, Predicted species name, Tier level, Combined score, Geo score, HTML-formatted view link to the original iMap image for human review.

Notes:

- The HTML view links allow direct visualization of the reported image without downloading.
- The combined threshold logic helps balance automated confidence with geographic likelihood and prioritization by Tier.
- NY iMapInvasives data managers can use the final Excel file to validate records, escalate questionable cases, and flag for expert review.

7.2. Threshold Estimation

Download “Threshold estimation.ipynb” to determine optimal thresholds for identifying high-confidence invasive species records using a logistic regression model. This notebook is part of the iMapML pipeline and uses prediction outputs from VisionAPI along with metadata scores (e.g., geo scores) to define robust cutoffs for combined scores. These thresholds help categorize records for auto-confirmation, human review.

General Workflow and Instructions for use:

This notebook evaluates prediction confidence using logistic regression and computes performance metrics like AUC, ROC, Youden’s J Statistics and precision-recall. It consists of the following key steps:

- a) **Import Required Libraries:** The notebook loads Python libraries such as pandas, numpy, scipy, seaborn, matplotlib, and sklearn for statistical modeling, plotting, and data analysis.
- b) **Set Working Directory and Load Data:** The current directory is changed to the working folder (e.g., "iMapML/Outputs"), and the prediction dataset (e.g., vision_predictions_multiple_records.xlsx) is loaded.

- c) Apply Logistic Regression: Logistic regression with Youden's J statistics is used to identify thresholds.
- d) Plot ROC and Precision-Recall Curves: ROC and PR curves are plotted with AUC values to visually assess model performance. These curves help in selecting the best decision threshold (cutoff value) for classification.
- e) Threshold Analysis: A range of thresholds is applied to calculate true positive rate (TPR), false positive rate (FPR), and precision/recall. A recommended threshold is identified where precision and recall are balanced (or based on desired sensitivity).

Notes:

- This notebook helps you define the optimal score thresholds for auto-confirming invasive species identifications.
- The combined `_score` used here is derived from VisionAPI output, spatial (geo) relevance, and New York's invasive species Tier system.
- The output thresholds can be applied in both single and batch record processing to filter records for expert review or acceptance.

8. Conclusion

This manual provides a concise and practical guide to using the iMapML tool for invasive species identification in New York State. From installation and environment setup to species prediction and threshold analysis, each step is designed to support accurate and efficient data processing. By integrating machine learning, geospatial intelligence, and the NYS Tier system, the tool empowers data managers to streamline species confirmation and improve decision-making in invasive species management.